

BÀI GIẢNG KHAI PHÁ DỮ LIỆU WEB

CHƯƠNG 4.

TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

NGUYỄN THÀNH THỦY

BỘ MÔN TIN HỌC QUẢN LÝ

TRƯỜNG ĐẠI HỌC KINH TẾ, ĐẠI HỌC ĐÀ NẴNG

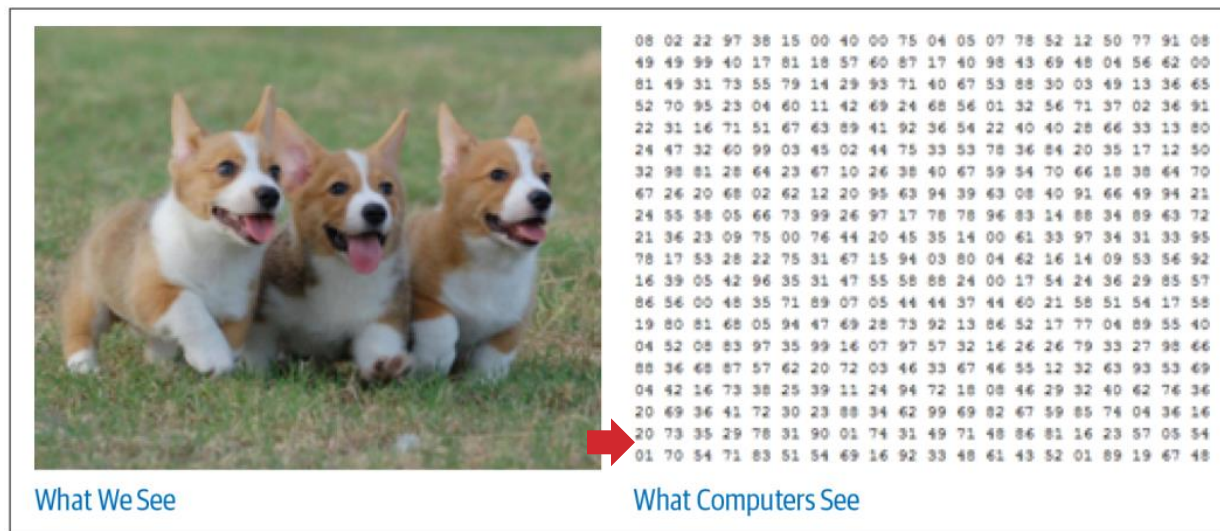
THUYNT@DUE.EDU.VN

NỘI DUNG

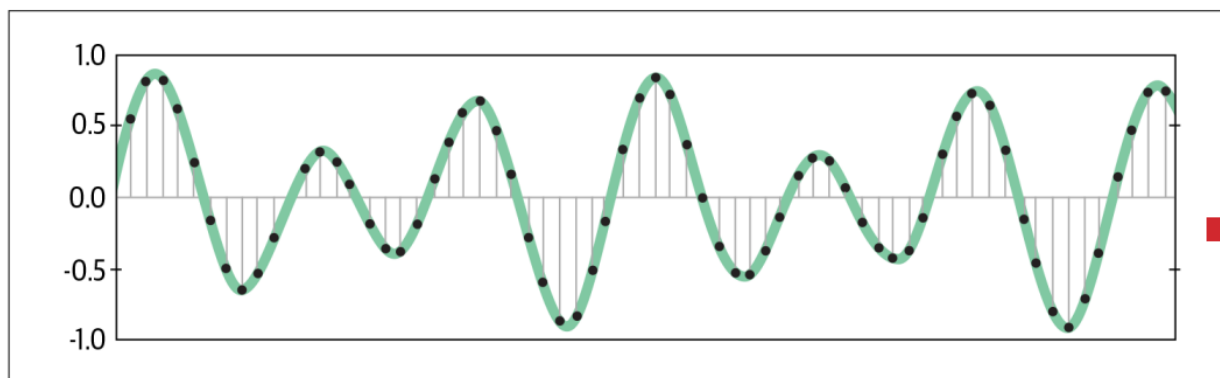
- ❑ Tổng Quan Về Trích Xuất Đặc Trưng Văn Bản
- ❑ Phương Pháp Trích Xuất Đặc Trưng Văn Bản

TỔNG QUAN VỀ TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Khái niệm



How we see an image versus how computers see it

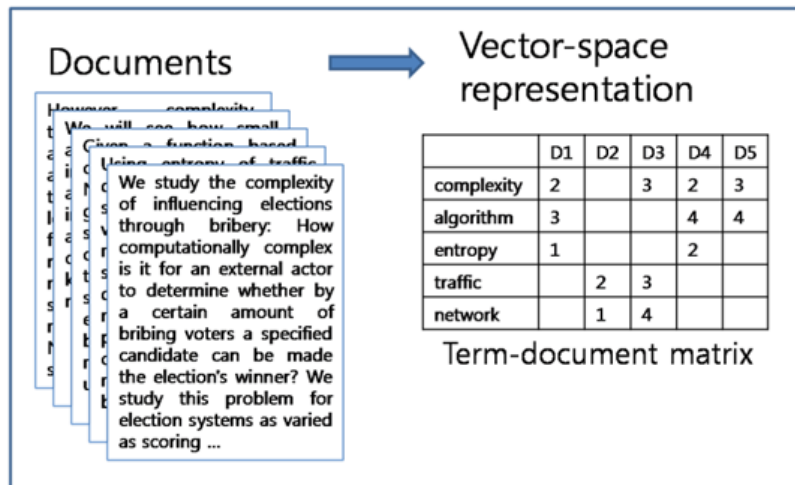


[-1274, -1252, -1160, -986, -792, -692, -614, -429, -286, -134, -57, -41, -169, -456, -450, -541, -761, -1067, -1231, -1047, -952, -645, -489, -448, -397, -212, 193, 114, -17, -110, 128, 261, 198, 390, 461, 772, 948, 1451, 1974, 2624, 3793, 4968, 5939, 6057, 6581, 7302, 7640, 7223, 6119, 5461, 4820, 4353, 3611, 2740, 2004, 1349, 1178, 1085, 901, 301, -262, -499, -488, -707, -1406, -1997, -2377, -2494, -2605, -2675, -2627, -2500, -2148, -1648, -970, -364, 13, 260, 494, 788, 1011, 938, 717, 507, 323, 324, 325, 350, 103, -113, 64, 176, 93, -249, -461, -606, -909, -1159, -1307, -1544]

Speech signal represented by a numerical vector

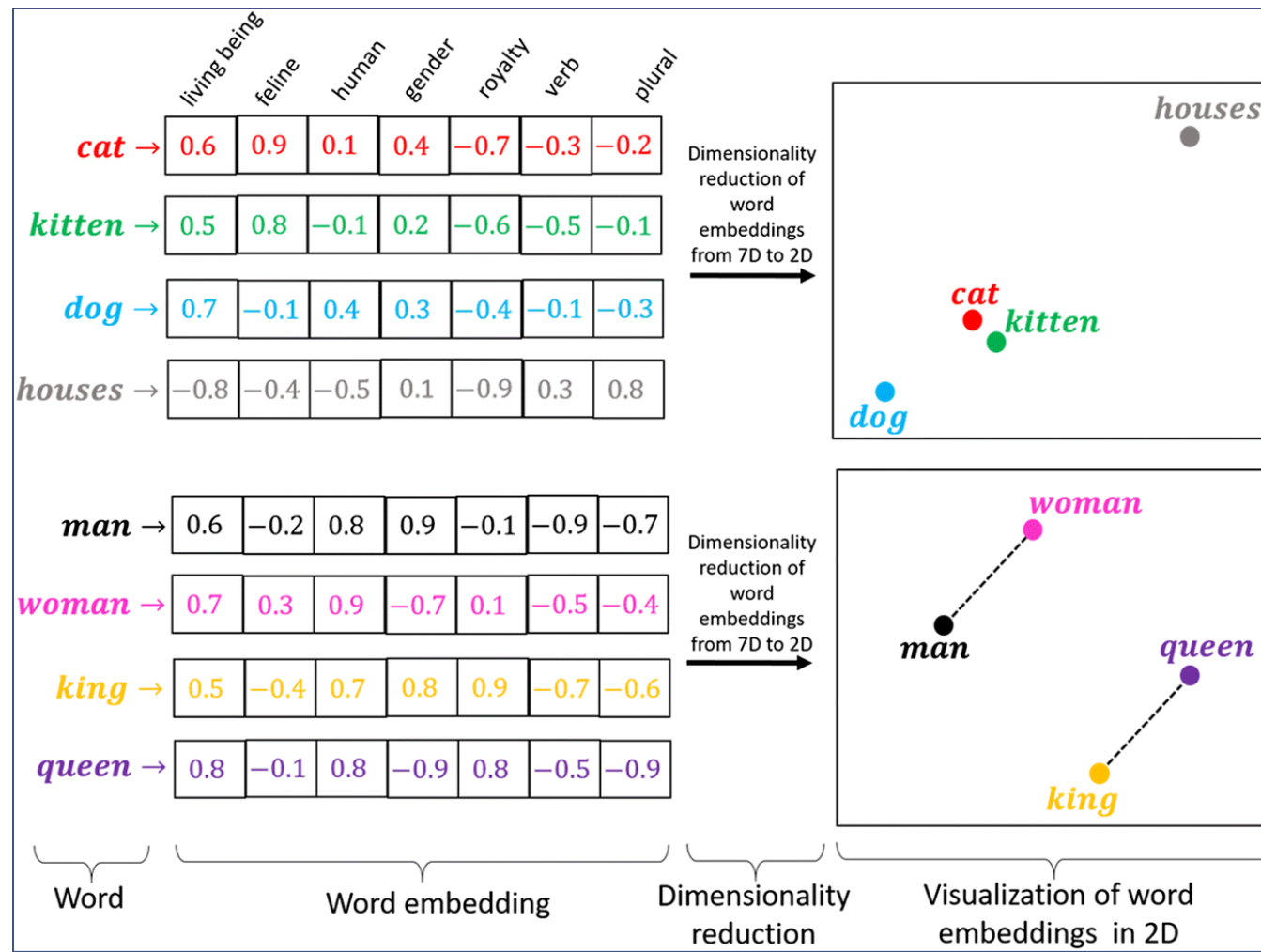
TỔNG QUAN VỀ TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Khái niệm



Bag of Words

Source: <https://qphs.fs.quoracdn.net/main-qimg-27639a9e2f88baab88a2c575a1de2005>



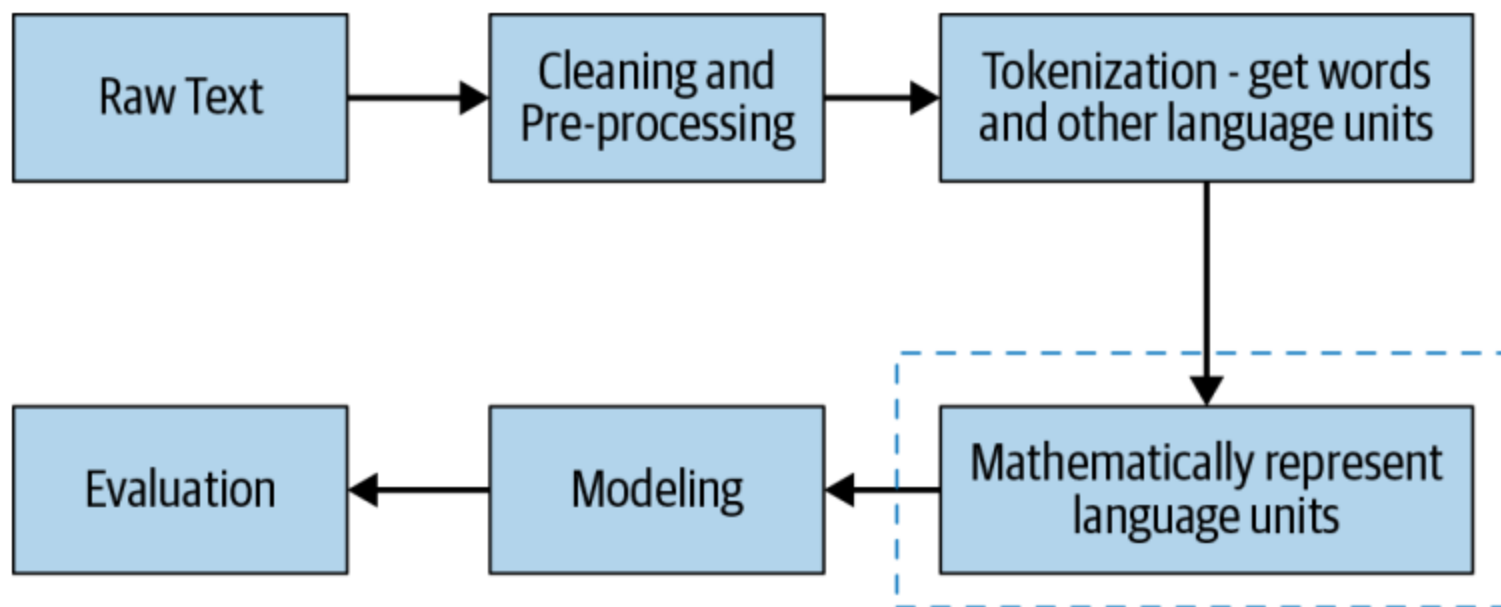
Word Embedding

Source: https://miro.medium.com/max/1400/1*sAJdxEsDjsPMioHyzlN3_A.png

TỔNG QUAN VỀ TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Khái niệm

- Trích xuất đặc trưng văn bản - Text Feature Extraction;
- Là quá trình chuyển đổi dữ liệu văn bản thô thành các đặc trưng có thể sử dụng trong các mô hình học máy, phân tích dữ liệu, hoặc xử lý ngôn ngữ tự nhiên;
- Nhằm tạo ra một biểu diễn số hóa của văn bản, giữ lại thông tin quan trọng và loại bỏ những thông tin không cần thiết;



Scope of this chapter within the NLP pipeline

TỔNG QUAN VỀ TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Các phương pháp

- One-Hot Encoding;
- Bag-of-Words (BOW);
- N-grams (Bag-of-N-grams);
- TF-IDF (Term Frequency-Inverse Document Frequency);
- Word2Vec (Word Embedding);

Lựa chọn phương pháp trích xuất đặc trưng **phụ thuộc vào bài toán và dữ liệu** đang giải quyết. Trong nhiều trường hợp, **kết hợp nhiều phương pháp trích xuất đặc trưng cũng mang lại kết quả tốt hơn.**

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ One-Hot Encoding

- Trong NLP, One-hot Encoding được sử dụng để biểu diễn **các từ dưới dạng vector**;
- **Mỗi từ được biểu diễn bằng một vector** có kích thước bằng với số lượng từ khác nhau trong văn bản;
- Vector có giá trị **1** tại vị trí tương ứng với từ đó, và giá trị **0** ở các vị trí còn lại;
- Kỹ thuật này không thể đếm được tần suất xuất hiện của một từ, dẫn đến mất thông tin.

Ví dụ 4.1, đoạn văn bản

“We are learning Natural Language Processing”

Được biểu diễn bởi vector 6x6 như sau:

	0	1	2	3	4	5
We	1	0	0	0	0	0
are	0	1	0	0	0	0
learning	0	0	1	0	0	0
Natural	0	0	0	1	0	0
Language	0	0	0	0	1	0
Processing	0	0	0	0	0	1

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ One-Hot Encoding

■ Ưu điểm

- Dễ hiểu và thực hiện;
- Không có sự mất mát thông tin;
- Tính đa dạng;

■ Hạn chế

- Không có thông tin về mối quan hệ giữa các từ;
- Kích thước vector lớn;
- Không phù hợp cho các tác vụ đòi hỏi hiểu nghĩa của từ;

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ One-Hot Encoding

▪ Ví dụ 4.1

```
import pandas as pd

# Danh sách các từ trong văn bản
text="We are learning Natural Language Processing"
words=text.split()

# Tạo DataFrame từ danh sách các từ
df = pd.DataFrame({'word': words})

# Thực hiện one-hot encoding
one_hot = pd.get_dummies(df['word'])

# Ghép kết quả vào DataFrame gốc
df = pd.concat([df, one_hot], axis=1)

print(df)
```



	word	Language	Natural	Processing	We	are	learning
0	We	0	0	0	1	0	0
1	are	0	0	0	0	1	0
2	learning	0	0	0	0	0	1
3	Natural	0	1	0	0	0	0
4	Language	1	0	0	0	0	0
5	Processing	0	0	1	0	0	0

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ One-Hot Encoding

- **Bài tập**, xây dựng vector One-Hot Encoding cho bộ dữ liệu sau
 - d1="both good and distinctive"
 - d2="good but not distinctive"
 - d3="distinctive but not good, or"
 - d4="neither good nor distinctive"

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Bag-of-Word (BOW)

- Bag-of-Word (BOW / Túi từ), còn gọi là kỹ thuật **Count Vectorizer**;
- Chuyển **một đoạn văn bản thành một vector**, dựa trên tần suất xuất hiện của từ trong toàn bộ văn bản;
- Là một dạng mở rộng của One-Hot Encoding;
- **Ví dụ**, xét khối văn bản sau:

Good movie

Not a Good movie

Did not like

	good	movie	not	a	did	like
good movie	1	1	0	0	0	0
not a good movie	1	1	1	1	0	0
did not like	0	0	1	0	1	1

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Bag-of-Word (BOW)

- Ví dụ, xây dựng vector BOW cho 2 câu văn bản sau :

Câu 1: “The quick brown fox jumps over the lazy dog and”

Câu 2: “Never jump over the lazy dog quickly”

Từ 2 câu trên, tiến hành xây dựng từ điển chứa các từ xuất hiện trong từng câu.

$W = \{\text{'and', 'brown', 'dog', 'fox', 'jump', 'lazy', 'never', 'over', 'quick', 'quickly', 'the'}\}$

Sentences	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
The quick brown fox jumps over the lazy dog and	1	1	1	1	0	1	1	0	1	1	0	2
Never jump over the lazy dog quickly	0	0	1	0	1	0	1	1	1	0	1	1

○ Trong Câu 1:

- Từ “the” xuất hiện 2 lần, nên phần tử thứ 11 sẽ mang giá trị 2;
- Các từ “and”, “brown”, “dog”, “fox”, “jump”, “lazy”, “over”, “quick” xuất hiện 1 lần;
- Vector chỉ mang giá trị 0, khi từ đó không xuất hiện trong câu như “never” và “quickly”;

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Bag-of-Word (BOW)

- Ví dụ, xây dựng vector BOW cho 2 câu văn bản sau :

Câu 1: “you have no dog”

Câu 2: “no, you have dog”

Từ 2 câu trên, tiến hành xây dựng từ điển chứa các từ xuất hiện trong từng câu.

$W = \{\text{'dog'}, \text{'have'}, \text{'no'}, \text{'you'}\}$

Sentences	dog	have	no	you
you have no dog	1	1	1	1
no, you have dog	1	1	1	1

- Hai câu trên có nội dung khác nhau, nhưng có cùng vector.

→ Vector BOW có hạn chế, đó là **không phân biệt thứ tự của các từ trong câu**.

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Bag-of-Word (BOW)

- Ví dụ 4.2a, code python thuần
 - Source Code: https://github.com/thuy-nguyenthanh/Web_Mining_Class/blob/main/Chapter4/Exam42a.py
- Ví dụ 4.2b, sử dụng thư viện sklearn
 - Source Code: https://github.com/thuy-nguyenthanh/Web_Mining_Class/blob/main/Chapter4/Exam42b.py

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

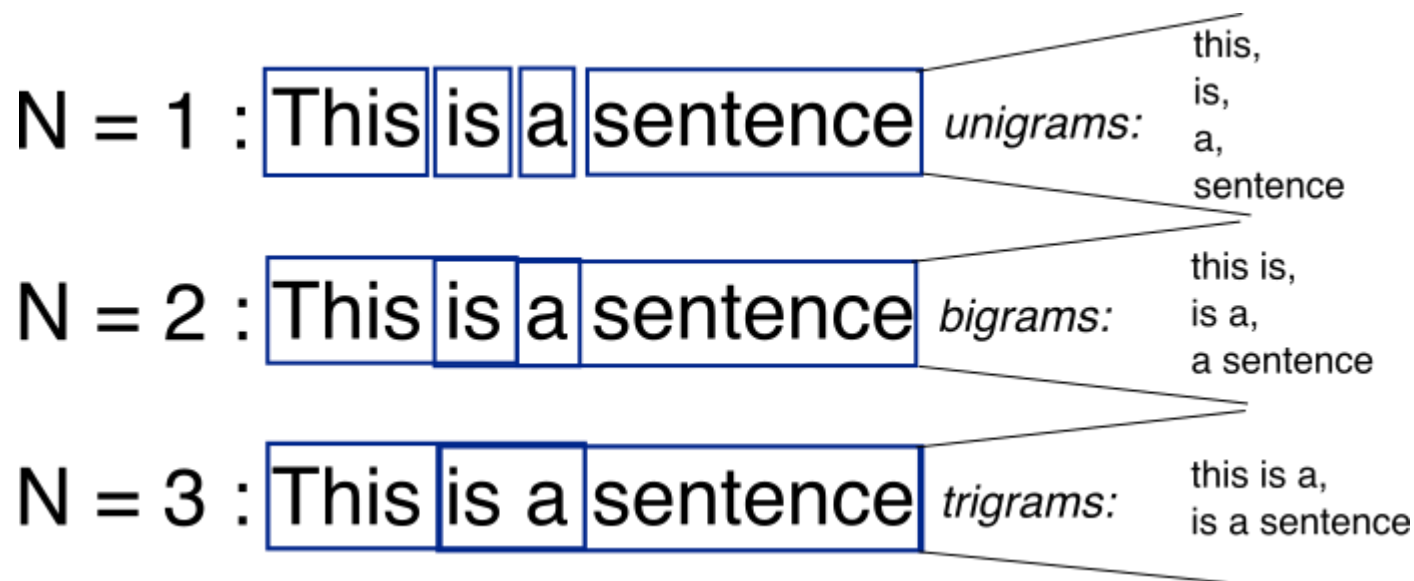
❑ Bag-of-Word (BOW)

- **Bài tập**, xây dựng vector BOW cho bộ dữ liệu sau:
d1="both good and distinctive"
d2="good but not distinctive"
d3="distinctive but not good, or"
d4="neither good nor distinctive"

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ N-grams

- **N-grams** hay còn gọi là **Bag-of-N-gram**, là phương pháp mở rộng của Bag-of-Words;
- **n** trong **n-grams** biểu thị số lượng các từ liên tiếp trong chuỗi;
 - Ví dụ, với $n=2$, ta có mô hình bigram, với $n=3$, ta có mô hình trigram,...
- Dữ liệu n-grams là dữ liệu đầu vào cho mô hình n-grams, được sử dụng rộng rãi trong nhiều ứng dụng NLP.



PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

- **Ví dụ**, xây dựng vector đặc trưng cho 2 câu sau với **1-grams** và **2-grams**

Doc1 = "The quick brown fox jumps over the lazy dog and"

Doc2 = "Never jump over the lazy dog quickly"

- **Bước 1: xây dựng N-grams**

Doc1=('the', 'quick', 'brown', 'fox', 'jumps', 'over', 'lazy', 'dog', 'and', 'the quick', 'quick brown', 'brown fox', 'fox jumps', 'jumps over', 'over the', 'the lazy', 'lazy dog', 'dog and')

Doc2=('never', 'jump', 'over', 'the', 'lazy', 'dog', 'quickly', 'never jump', 'jump over', 'over the', 'the lazy', 'lazy dog', 'dog quickly')

- **Bước 2: xây dựng tập từ vựng**, có 24 từ (được sắp xếp theo thứ tự alphabet)
(and, brown, brown fox, dog, dog and, dog quickly, fox, fox jumps, jump, jump over, jumps, jumps over, lazy, lazy dog, never, never jump, over, over the, quick, quick brown, quickly, the, the lazy, the quick)

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

- Ví dụ, xây dựng vector đặc trưng cho 2 câu sau với **1-grams** và **2-grams**

Doc1 = "The quick brown fox jumps over the lazy dog and"

Doc2 = "Never jump over the lazy dog quickly"

- **Bước 3: tạo vector đặc trưng**

	and	brown	brown fox	dog	dog and	dog quickly	fox	fox jumps	jump	jump over	jumps	jumps over
The quick brown fox jumps over the lazy dog and	1	1	1	1	1	0	1	1	0	0	1	1
Never jump over the lazy dog quickly	0	0	0	1	0	1	0	0	1	1	0	0
	lazy	lazy dog	never	never jump	over	over the	quick	quick brown	quickly	the	the lazy	the quick
The quick brown fox jumps over the lazy dog and	1	1	0	0	1	1	1	1	0	2	1	1
Never jump over the lazy dog quickly	1	1	1	1	1	1	0	0	1	1	1	0

➔ Doc1: [1 1 1 1 1 0 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 2 1 1]
Doc2: [0 0 0 1 0 1 0 0 1 1 0 0 1 1 1 1 1 1 0 0 1 1 1 0]

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

- Ví dụ 4.3a, tạo n-grams sử dụng Python thuần

```
def generate_ngrams(text, n):  
    # Tách các từ trong câu  
    words = text.split()  
  
    # Sử dụng hàm zip để tạo các cặp liên tiếp trong danh sách các từ  
    ngrams = zip(*[words[i:] for i in range(n)])  
  
    # Sử dụng list comprehension để tạo ra các n-grams  
    return [" ".join(gram) for gram in ngrams]  
  
text = "this is a test sentence"  
print(generate_ngrams(text, 2))
```



```
['this is', 'is a', 'a test', 'test sentence']
```

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

- Ví dụ 4.3b, tạo n-grams sử dụng thư viện nltk

```
import nltk
from nltk.util import ngrams

# Function to generate n-grams from sentences.
def extract_ngrams(data, num):
    n_grams = ngrams(nltk.word_tokenize(data), num)
    return [ ' '.join(grams) for grams in n_grams ]

data = 'this is a test sentence'

print("1-gram: ", extract_ngrams(data, 1))
print("2-gram: ", extract_ngrams(data, 2))
print("3-gram: ", extract_ngrams(data, 3))
print("4-gram: ", extract_ngrams(data, 4))
```



```
1-gram: ['this', 'is', 'a', 'test', 'sentence']
2-gram: ['this is', 'is a', 'a test', 'test sentence']
3-gram: ['this is a', 'is a test', 'a test sentence']
4-gram: ['this is a test', 'is a test sentence']
```

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

- Ví dụ 4.3c, tạo vector đặc trưng n-grams sử dụng thư viện sklearn

```
from sklearn.feature_extraction.text import CountVectorizer
# Khởi tạo đối tượng CountVectorizer
# Tham số ngram_range=(1,2) cho biết chúng ta muốn sử dụng N-grams từ 1 đến 2.
vectorizer = CountVectorizer(ngram_range=(1,2))

# Tạo ra một list chứa các câu cần biểu diễn
Data = ["The quick brown fox jumps over the lazy dog and",
        "Never jump over the lazy dog quickly"]

# Biểu diễn vector đặc trưng cho các câu
features = vectorizer.fit_transform(Data)

# In ra các feature vector tương ứng với các câu
print(features.toarray())
```

➡

```
[[1 1 1 1 1 0 1 1 0 0 1 1 1 1 0 0 1 1 1 1 0 2 1 1]
 [0 0 0 1 0 1 0 0 1 1 0 0 1 1 1 1 1 1 1 0 0 1 1 1 0]]
```

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

- Ví dụ 4.3d, tạo n-grams sử dụng thư viện sklearn đo độ tương đồng

```
from sklearn.feature_extraction.text import CountVectorizer
from scipy.spatial.distance import euclidean
```

```
texts=['you have no dog', 'no, you have dog', 'you have a dog']
```

```
# bigram
```

```
bigram = CountVectorizer(ngram_range = (2, 2))
n1, n2, n3 = bigram.fit_transform(texts).toarray()
```

```
# trigram
```

```
trigram = CountVectorizer(ngram_range = (3, 3))
n1, n2, n3 = trigram.fit_transform(texts).toarray()
```

```
#Sử dụng Euclidean để đo khoảng cách giữa các vector
```

```
print(euclidean(n1, n2), euclidean(n2, n3), euclidean(n1, n3))
```

```
#--> Cặp câu (n2,n3) có độ tương đồng cao nhất
```

➡ 2.0 1.0 1.7320508075688772

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ N-grams

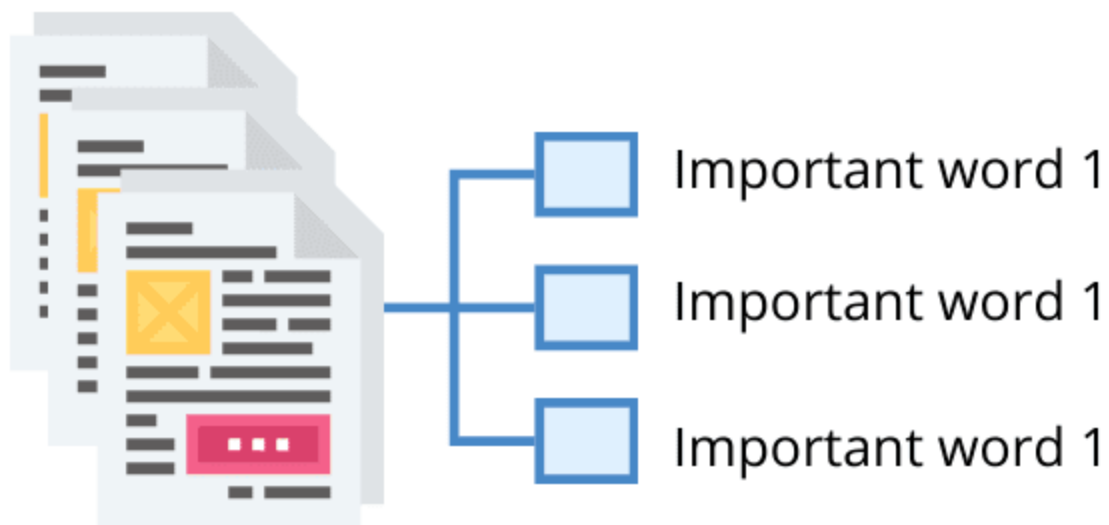
- **Bài tập**, xây dựng vector đặc trưng 1-grams, 2-grams cho bộ dữ liệu sau:
d1="both good and distinctive"
d2="good but not distinctive"

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ TF-IDF

- **TF-IDF**: Term Frequency – Inverse Document Frequency, **giúp xác định độ quan trọng của một từ trong văn bản**;
- **TF**: Tần suất xuất hiện của một từ trong văn bản;
- **IDF**: Tần số nghịch đảo của 1 từ trong tập văn bản (corpus);

TF - IDF



Đọc thêm:

- <https://medium.com/nlplanet/two-minutes-nlp-learn-tf-idf-with-easy-examples-7c15957b4cb3>
- <https://towardsdatascience.com/natural-language-processing-feature-engineering-using-tf-idf-e8b9d00e7e76>

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

$$\text{TF-IDF}(\mathbf{w}_k, \mathbf{d}_i, \mathbf{D}) = \text{TF}(\mathbf{w}_k, \mathbf{d}_i) \times \text{IDF}(\mathbf{w}_k, \mathbf{D})$$

$$\text{TF}(\mathbf{w}_k, \mathbf{d}_i) = \frac{\text{Tổng số lần xuất hiện của từ } \mathbf{w}_k \text{ trong văn bản } \mathbf{d}_i}{\text{Tổng số từ trong văn bản } \mathbf{d}_i}$$

$$\text{IDF}(\mathbf{w}_k, \mathbf{D}) = \log \left(\frac{\text{Tổng số câu văn bản trong tập } \mathbf{D}}{\text{Tổng số câu văn bản trong tập } \mathbf{D} \text{ có chứa từ } \mathbf{w}_k} \right)$$

Trong đó:

- **D**: là tập corpus, $\mathbf{D} = \{\mathbf{d}_1, \mathbf{d}_2, \mathbf{d}_3, \dots, \mathbf{d}_n\}$;
- **d_i**: là một câu văn bản trong tập **D**;
- **w_k**: là một từ, trong bộ từ vựng **W** trong tập **D**, $\mathbf{W} = \{\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3, \dots, \mathbf{w}_m\}$

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

d_1 = "the quick brown fox jumps over the lazy dog and"

d_2 = "never jump over the lazy dog quickly"

$D = \{d_1, d_2\}$

Bước 1, Xây dựng tập từ vựng có trong corpus:

$W = \{\text{and, brown, dog, fox, jump, jumps, lazy, never, over, quick, quickly, the}\}$

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 2, Tính TF:

- Tính $TF(w_k, d_1)$:

- **Bước 2.1:** Xác định số lần xuất hiện của các từ w_k trong câu d_1 ;

d_1	"the quick brown fox jumps over the lazy dog and"											
w_k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
Số lần xuất hiện	1	1	1	1	0	1	1	0	1	1	0	2

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 2, Tính TF:

- Tính $TF(w_k, d_1)$:

- **Bước 2.1:** Xác định số lần xuất hiện của các từ w_k trong câu d_1 ;
- **Bước 2.2:** Xác định tổng số từ trong câu d_1

Câu d_1 = "the quick brown fox jumps over the lazy dog and"

→ có **10** từ.

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 2, Tính TF:

- Tính $TF(w_k, d_1)$:

- **Bước 2.1:** Xác định số lần xuất hiện của các từ w_k trong câu d_1 ;
- **Bước 2.2:** Xác định Tổng số từ trong câu d_1 ;
- **Bước 2.3:** tính $TF(w_k, d_i)$

$$TF(w_k, d_i) = \frac{\text{Tổng số lần xuất hiện của từ } w_k \text{ trong văn bản } d_i}{\text{Tổng số từ trong văn bản } d_i}$$

$$TF(\text{"and"}, d_1) = 1/10 = 0.1$$

$$TF(\text{"brown"}, d_1) = 1/10 = 0.1$$

$$TF(\text{"dog"}, d_1) = 1/10 = 0.1$$

$$TF(\text{"fox"}, d_1) = 1/10 = 0.1$$

$$TF(\text{"jump"}, d_1) = 0/10 = 0$$

...

$$TF(\text{"the"}, d_1) = 2/10 = 0.2$$

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 2, Tính TF:

- Tính $TF(w_k, d_1)$:

- **Bước 2.1:** Xác định số lần xuất hiện của các từ w_k trong câu d_1 ;
- **Bước 2.2:** Xác định Tổng số từ trong câu d_1 ;
- **Bước 2.3:** tính $TF(w_k, d_i)$

d_1	"the quick brown fox jumps over the lazy dog and"											
w_k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
Số lần xuất hiện	1	1	1	1	0	1	1	0	1	1	0	2
$TF(w_k, d_1)$	0.1	0.1	0.1	0.1	0	0.1	0.1	0	0.1	0.1	0	0.2

- **Bước 2.4:** lặp lại **Bước 2.1** cho đến khi hết tập D;

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 2, Tính TF:

d_1 = "the quick brown fox jumps over the lazy dog and"

d_2 = "never jump over the lazy dog quickly"

d_1	"the quick brown fox jumps over the lazy dog and"											
w_k	and	brown	dog	fox	jump	jumps	Lazy	never	over	quick	quickly	the
Số lần xuất hiện	1	1	1	1	0	1	1	0	1	1	0	2
$TF(w_k, d_1)$	0.1	0.1	0.1	0.1	0	0.1	0.1	0	0.1	0.1	0	0.2
d_2	"never jump over the lazy dog quickly"											
w_k	and	brown	dog	fox	jump	jumps	azy	never	over	quick	quickly	the
Số lần xuất hiện	0	0	1	0	1	0	1	1	1	0	1	1
$TF(w_k, d_2)$	0	0	0.143	0	0.143	0	0.143	0.143	0.143	0	0.143	0.143

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 3, Tính IDF:

Bước 3.1: Tính số câu văn bản trong tập D: là **2** câu

Bước 3.2: Tính số câu văn bản trong tập D có chứa từ w_k

d_1 = "the quick brown fox jumps over the lazy dog and"

d_2 = "never jump over the lazy dog quickly"

w_k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
d_1	1	1	1	1	0	1	1	0	1	1	0	2
d_2	0	0	1	0	1	0	1	1	1	0	1	1
Số câu xuất hiện w_k	1	1	2	1	1	1	2	1	2	1	1	2

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 3, Tính IDF:

Bước 3.1: Tính số câu văn bản trong tập D: là **2** câu

Bước 3.2: Tính số câu văn bản trong tập D có chứa từ w_k

Bước 3.3: Tính $IDF(w_k, D)$ cho tất cả các từ w_k trong tập từ vựng W

$$IDF(w_k, D) = \log \left(\frac{\text{Tổng số câu văn bản trong tập D}}{\text{Tổng số câu văn bản trong tập D có chứa từ } w_k} \right)$$

w_k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
d_1	1	1	1	1	0	1	1	0	1	1	0	2
d_2	0	0	1	0	1	0	1	1	1	0	1	1
Số câu xuất hiện w_k	1	1	2	1	1	1	2	1	2	1	1	2
$IDF(w_k, D)$	0.301	0.301	0	0.301	0.301	0.301	0	0.301	0	0.301	0.301	0

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 4, Tính TF-IDF:

$$\text{TF-IDF}(w_k, d_i, D) = \text{TF}(w_k, d_i) \times \text{IDF}(w_k, D)$$

d1	"the quick brown fox jumps over the lazy dog and"											
w _k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
TF(w _k ,d ₁)	0.1	0.1	0.1	0.1	0	0.1	0.1	0	0.1	0.1	0	0.2
IDF(w _k ,D)	0.301	0.301	0	0.301	0.301	0.301	0	0.301	0	0.301	0.301	0
TF-IDF(w _k ,d ₂ ,D)	0.03	0.03	0	0.03	0	0.03	0	0	0	0.03	0	0
d2	"never jump over the lazy dog quickly"											
w _k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
TF(w _k ,d ₂)	0	0	0.143	0	0.143	0	0.143	0.143	0.143	0	0.143	0.143
IDF(w _k ,D)	0.301	0.301	0	0.301	0.301	0.301	0	0.301	0	0.301	0.301	0
TF-IDF(w _k ,d ₂ ,D)	0	0	0	0	0.043	0	0	0.043	0	0	0.043	0

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- Ví dụ, xác định TF-IDF của các từ trong bộ dữ liệu sau:

Bước 4, Tính TF-IDF:

Vector TF-IDF của các câu trong corpus

w_k	and	brown	dog	fox	jump	jumps	lazy	never	over	quick	quickly	the
d_1	0.03	0.03	0	0.03	0	0.03	0	0	0	0.03	0	0
d_2	0	0	0	0	0.043	0	0	0.043	0	0	0.043	0

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- **Ví dụ 4.4a**, xây dựng vector TF-IDF bằng Python thuần
 - Source Code: https://github.com/thuy-nguyenthanh/Web_Mining_Class/blob/main/Chapter4/Exam44a.py
- **Ví dụ 4.4b**, xây dựng vector TF-IDF bằng thư viện nltk
 - Source Code: https://github.com/thuy-nguyenthanh/Web_Mining_Class/blob/main/Chapter4/Exam44b.py

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ TF-IDF

- **Bài tập**, thực hiện các bước để xây dựng vector TF-IDF cho tập corpus sau:
d1="both good and distinctive"
d2="good but not distinctive"
d3="distinctive but not good, or"
d4="neither good nor distinctive"

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ Word2Vec

- **Word2Vec** còn gọi là **Word Embedding**, là một mô hình vector hóa văn bản. là phương pháp được Google công bố vào năm 2013;
 - Ý tưởng cơ bản của word2vec:
 - Hai từ xuất hiện trong những văn cảnh giống nhau, thường có ý nghĩa gần với nhau;
 - Ta có thể đoán được một từ nếu biết các từ xung quanh nó trong câu;
 - Ví dụ, với câu “**Hà Nội là ... của Việt Nam**” thì từ trong dấu ... khả năng cao là “**thủ đô**”.
- ✓ Từ “**thủ đô**” được gọi là **target word** hay **từ đích**;
 - ✓ Những từ xung quanh nó được gọi là **context words** hay **từ ngữ cảnh**;
 - ✓ Với mỗi từ đích và từ ngữ cảnh trong cùng một câu, **có vị trí cách từ đích một khoảng không quá $C/2$** , với **C** là một số tự nhiên dương.

Source:

- https://machinelearningcoban.com/tabml_book/ch_embedding/word2vec.html
- https://d2l.ai/vn/chapter_natural-language-processing-pretraining/word2vec_vn.html

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

❑ Word2Vec

- Xét ví dụ với câu: “The quick brown fox jump over the lazy dog”, với $C=4$

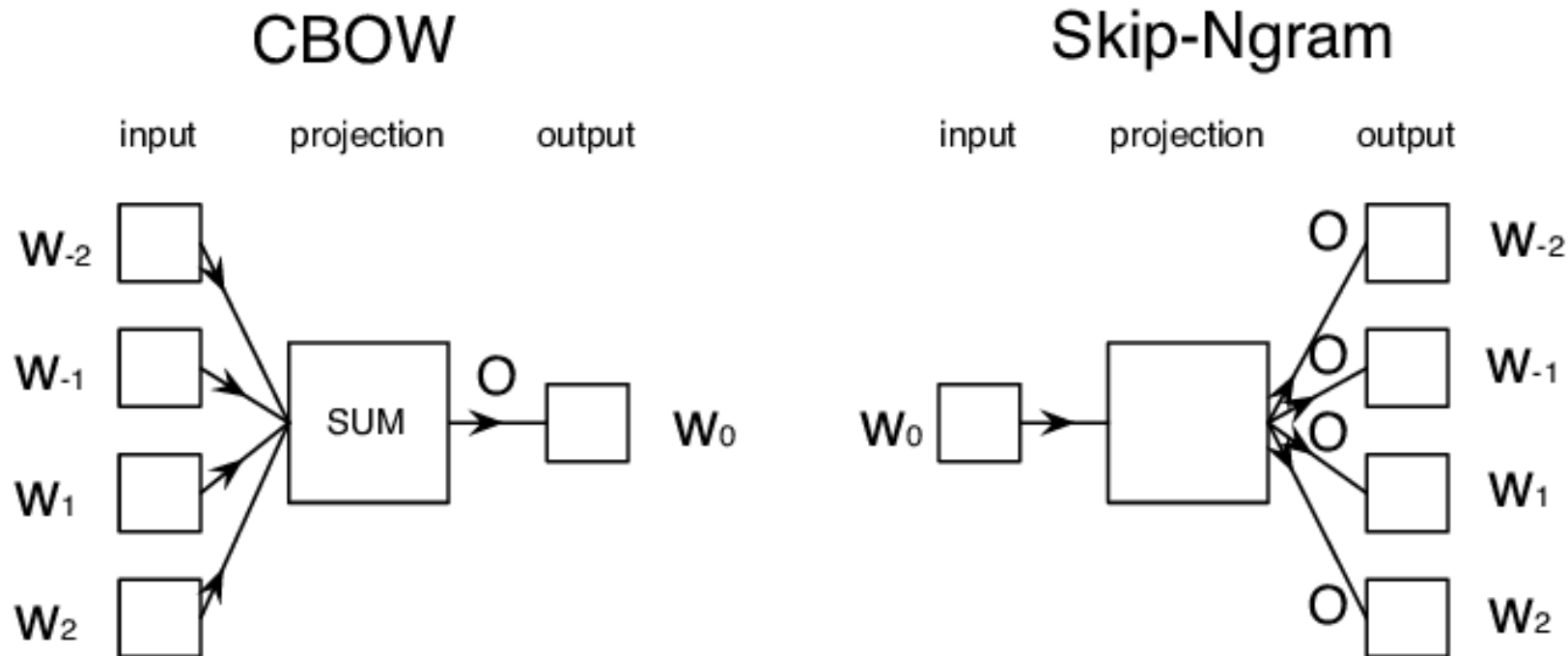
Source Text	Training Samples
The quick brown fox jumps over the lazy dog.	(the, quick) (the, brown)
The quick brown fox jumps over the lazy dog.	(quick, the) (quick, brown) (quick, fox)
The quick brown fox jumps over the lazy dog.	(brown, the) (brown, quick) (brown, fox) (brown, jumps)
The quick brown fox jumps over the lazy dog.	(fox, quick) (fox, brown) (fox, jumps) (fox, over)

Source: https://machinelearningcoban.com/tabml_book/ch_embedding/word2vec.html

PHƯƠNG PHÁP TRÍCH XUẤT ĐẶC TRƯNG VĂN BẢN

□ Word2Vec

- Có hai kỹ thuật xây dựng mô hình Word2vec:
 - **Skip-gram**: Dựa vào **từ đích** để dự đoán các **từ ngữ cảnh**;
 - **Continuous Bag of Words (CBOW)**: Dựa vào những **từ ngữ cảnh** để dự đoán **từ đích**;



Source Code: https://github.com/thuy-nguyenthanh/Web_Mining_Class/blob/main/Chapter4/Exam45.py

Source: <https://vinbigdata.com/kham-pha/word-embedding-so-sanh-cac-cach-tiep-can-truyen-thong-va-hien-dai.html>

Bài giảng KHAI PHÁ DỮ LIỆU WEB - Nguyễn Thành Thủy, MIS Dept. - DUE